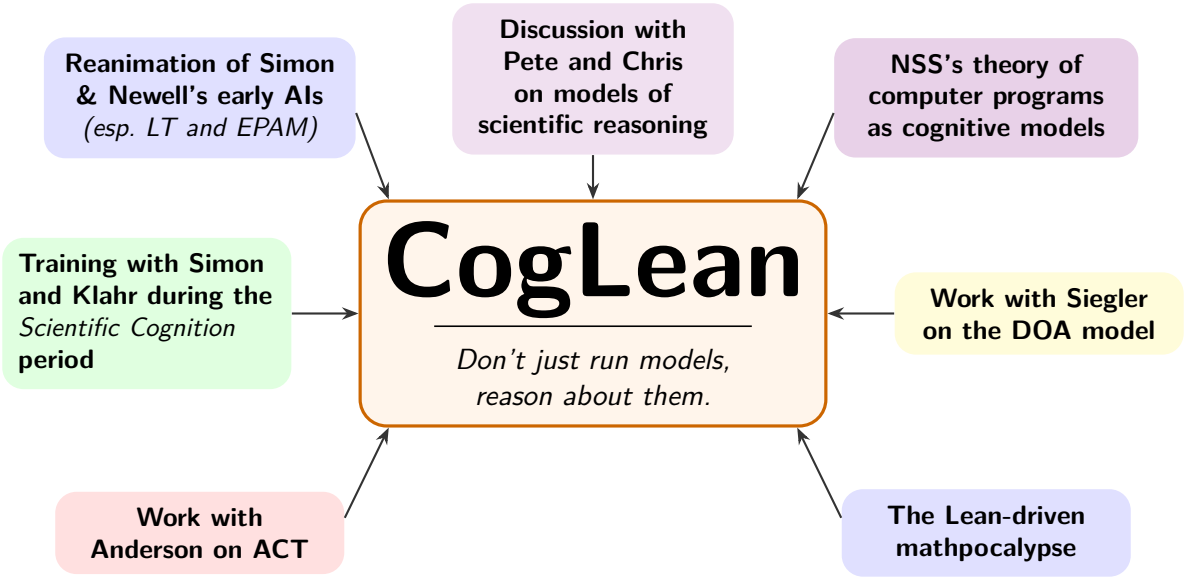
Epoofany


Reanimation of Simon & Newell's early AIs (esp. LT and EPAM)
Discussion with Pete and Chris on models of scientific reasoning
NSS's theory of computer programs as cognitive models
Training with Simon and Klahr during the Scientific Cognition period
CogLean
Don't just run models, reason about them.
Work with Siegler on the DOA model
Work with Anderson on ACT
The Lean-driven mathpocalypse

CogLean

Towards a Cumulative, Machine-Checked Theory of Cognition

Jeff Shrager

October 2026

Draft for Comment

The problem: making theory cumulative

- Newell (1973): twenty questions with nature, answers never assembled; Newell (1990): unified theories.
- Cognitive architectures (Soar, ACT-R; now the Common Model) have spent decades on this: **shared mechanisms**, so models of different tasks reuse and integrate theoretical commitments.
- What they accumulate is mainly **models**: what a model predicts is established by running it, one setting at a time, and a result for one model is rarely reusable *as a result* by another.
- CogLean, **complementary**: accumulate machine-checked **theorems** about mechanisms, reusable deductively, and stated across architectural commitments.

Can independently developed theories, in a common formal library, be composed to derive meaningful consequences we'd otherwise need a new model to find?

Prior formal approaches to cognitive theory

Work	Formalism	What it establishes
Soar specified in Z (Milnes et al. 1992)	Z notation	a complete, low-level, non-executable specification of one architecture
Sceptic (Cooper, Fox, Farringdon & Shallice 1996)	executable specification language	models (Soar among them) stated in a language designed for cognitive models
Reconstructing ACT-R (Stewart & West 2007)	Python reimplementation	core commitments separated from implementation; alternative components
ACT-R operational semantics (Gall & Frühwirth 2018)	abstract semantics; CHR	a translation proved sound and complete; confluence tests
ACT-R as timed automata (Langenfeld et al. 2019, 2021)	timed automata	exhaustive defect analysis of models; the semantics proved correct
A common formal ground for architectures (Ragni et al. 2018)	formal framework	ACT-R and Soar embedded for comparison
Soar to PRISM (Ganeriwala et al. 2026)	Markov chains, PRISM	probabilistic model checking of stochastic models
Verbal theories as logic programs; as automata (Rohner & Kjellerstrand 2021; Finkel et al. 2025)	Prolog; automata	what a verbal theory's premises entail
Critical tests from core assumptions (Fisher et al. 2025)	proofs in prose over model classes	which processing structures ACT-R's core rules out

The analogy: Mathlib

- Mathlib, the community library of formalized mathematics in Lean, proposes no new mathematics.
- It is a **shared environment**: definitions made once, every result machine-checked, later work citing earlier results.
- A theorem about groups, once proved, is available to everyone who uses groups.

CogLean asks whether cognitive theory can be organized the same way.
It is not a new theory of cognition, nor a new modeling language.
And it is nowhere near Mathlib's scale: a first demonstration.

CogLean's layers

Applications: task models assembled from theories and library parts (three-digit addition)

Theories: EPAM, strategy choice (DOA), ACT-R declarative memory, concepts, semantic memory, ...

CogLib: representations, memories, processes, decision cycle; a retriever interface; theory-neutral phenomena and laws, with their theorems

Lean 4 (core only): the language and the proof checker; exact rational probabilities

Each layer imports only those below it, and inherits their theorems.

Four kinds of object, kept apart

1. **Mechanisms** and their postulates: definitions (how a net sorts a code; how retrieval samples).
2. **Phenomena**: predicates, stated without any mechanism. A predicate asserts nothing; it is a target.
3. **Theorems**: a mechanism produces behavior satisfying a predicate.
4. **Conditions**: situational ("x is confused with q"), empirical premises not yet explained ("associations are sharp"), well-formedness.

A theorem with a phenomenon among its hypotheses is a relation between phenomena, not a prediction. The catalog flags these mechanically.

Three capabilities

1. **Formalization**: theories as definitions, postulates and theorems, with every assumption explicit.
2. **Composition**: mechanisms reused in new contexts, and so are the *theorems* about them.
   - reusable algorithms: useful software;
   - reusable *results*: a contribution to theory.
3. **Discovery**, of a specific kind: consequences of proposed theories that their authors didn't state; unstated assumptions; results for whole classes of models; where theories diverge. (Not new hypotheses: see the end.)

Two theories, formalized on their own

EPAM (Feigenbaum & Simon)

- A discrimination net sorts a code to a stored image.
- Learning grows a test only where codes are confused.
- Proved for every net: recognition depends only on tested features; learning never disturbs other items.

DOA (Siegler & Shrager 1984)

- Associations from problems to answers; sample, state if above a criterion; else fingers, then counting.
- Proved for every setting: retrieval used with probability $1 - (1 - R)^n$; counting errors independent of the associations.

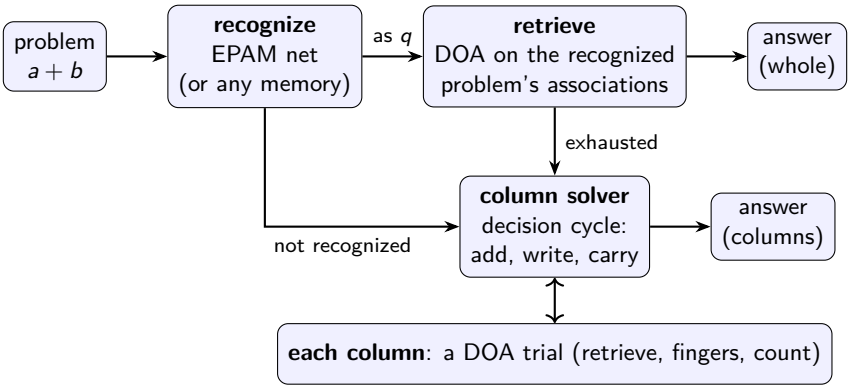
Formalization alone corrected a theory's claim

The DOA chapter: "a peaked distribution results in less frequent use of overt strategies."

- **Follows**: moving strength to the correct answer never raises retrieval errors, at any setting.
- **Does not follow**: it can *raise* overt strategy use.
  - $1 + 1$, strengths $.38, .31, .31 \rightarrow .39, .30, .31$;
  - averaged over the chapter's twelve criteria: $7/12 \rightarrow 73/120$.
  - The weakened wrong answer drops to a criterion, stops being stated, and the child turns to fingers.

The claim needs a condition the chapter doesn't state.

The application: three-digit addition



A demonstration, not a theory of arithmetic. In Lean, the composition is three definitions.

No answer supplied

- The earlier worked case used a backup that returned the correct sum: `Dist.pure p.sum`.
  - Enough to instantiate theorems about *every* backup;
  - but it supplies the answer instead of computing it.
- In the integrated model, **nothing is handed the answer**.
  - 250 for $150 + 100$ is assembled by the solver from three DOA trials and a carry rule, and is never certain.
  - Correct knowledge enters only as single-digit associations and the carry rule.

So composition has to work twice: the mechanisms run together, and their theorems transfer.

What actually happens when retrieval fails

1. **Recognize**: the EPAM net sorts the problem's code to a stored problem, possibly the wrong one.
2. **Retrieve**: DOA searches that problem's associations for an answer above the criterion.
3. **Fail**: if the search is exhausted, the backup is invoked.
4. **Solve**: the backup runs column addition on CogLib's general problem-solving machinery.
5. **Each column**: units, then tens, then hundreds; each digit sum is a separate DOA trial (retrieve, else fingers, else count).
6. **Combine**: each trial's *distribution* of answers feeds the rest of the calculation, with carrying and place value.
7. **Result**: a probability distribution over complete multi-digit answers, errors included.

`Dist.bind` composes these steps: it carries *every* possible outcome, with its probability, through the steps that follow. Nothing is sampled, and nothing looks up the right answer.

The general machinery, and its arithmetic instance

CogLib (`Process/ProblemSolving`), task-general:

- problem spaces: states; operators with preconditions and (probabilistic) effects;
- a **decision cycle**: a procedure proposes; a violated precondition is an *impasse*, fixed by repairs and critics;
- separately, **GPS**: goals, subgoals, backtracking.

MultiDigit (`Solver.lean`), the instance:

- operators: add a column, write, write and carry, finish;
- `full`: the procedure that proposes them in turn;
- `lenient`: the repair policy (no critics).

```
def solverBackup (F : Facts) (p : Problem) : Dist Nat :=
  Solver.answers F.fact Solver.full Solver.lenient p
```

`F.fact`: the DOA single-digit trial; `Solver.answers`: runs CogLib's decision cycle. The integrated model uses the **decision cycle**; GPS runs on the same space in `SolverGPS.lean` (units first; exact with exact facts) but is not part of it.

The bridge: two executable descriptions, one distribution

- `columns`: a simple, direct implementation of probabilistic column arithmetic.
- `answers`: the same task on CogLib's state-and-operator decision cycle.

```
theorem bridge (fact : Nat → Nat → Dist Nat) (p : Problem) :
    answers fact full lenient p = columns fact p
```

- **Proved**, not tested: identical answer distributions for *every* problem and *every* single-digit process.
- So every theorem about `columns` (place value, a column error as a single-digit error in place) holds for the solver.
- It equates *outputs*, not internal processes: the two still differ as accounts of what a child does.

How theorems compose

- **The bridge** (previous slide): every column theorem transfers to the solver.
- **Associativity**: every arrow is a two-stage draw; draws regroup, so a component's theorem holds inside the whole.
- **Well-formedness, inherited**: the composed trial is a probability distribution, proved by chaining lemmas from three packages:
  - retrieval, a DOA trial, the column procedure, the bridge;
  - total mass 1, *and* nonnegative weights.

These are library facts, not psychology; they are what let results transfer.

Confident errors

If q is recognized as itself, x is confused with q, q's associations are **sharp**, and their sums differ:

(i) whole-problem retrieval never errs on q;

(ii) every whole-problem answer to x is wrong;

(iii) x reaches the solver *exactly as often* as q.

- **Neither theory alone**: in DOA, errors and backup use come from one distribution; recognition gives x another problem's distribution.
- **Reused**: a one-line proof, from CogLib's theorem for *any* task with problems, a familiar set and a backup.
- **Conditions**: confusion (derived for learned nets); sharpness (assumed, not yet explained); positive strengths.

In words: the theorem on the next slide

CogLib's `confident_errors`, for *any* task, memory and backup:

- **Parameters**: how problems are coded, what memory calls up (`retrieve`), the associations (`str`), the criterion and search length, and the backup.
- **Hypotheses**:
  - strengths have positive totals; the backup is a proper distribution;
  - q calls up itself, for certain (`hq`);
  - x calls up q, for certain (`hx`): the confusion;
  - q's associations are sharp (`hs`); the correct answers differ (`hne`).
- **Conclusion**: the *confusion-error pattern*, a phenomenon defined once in CogLib, apart from any mechanism: three equations, (i)–(iii) of the last slide.
- **Proof**: three lines, citing three earlier lemmas.

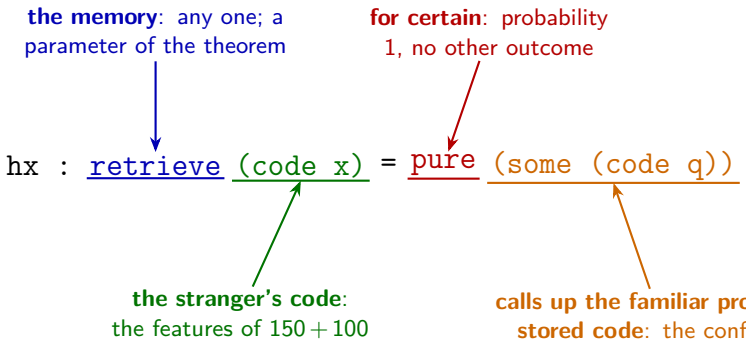
The theorem, in Lean

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    Phenomena.ConfusionErrorPattern (recognizeRetrieve code retrieve str as cc n backup)
      (wholeWrong correct) isWhole isBackup x q := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup
      correct q q (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup
      correct x _ _ (hT _) hx hs hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]

def ConfusionErrorPattern (trial : P → Dist O) (wholeWrong : P → O → Bool)
    (isWhole isBackup : O → Bool) (x q : P) : Prop :=
  prob (trial q) (wholeWrong q) = 0 ∧
    prob (trial x) (wholeWrong x) = prob (trial x) isWhole ∧
    prob (trial x) isBackup = prob (trial q) isBackup
```

Verbatim from CogLib (`Process/RecognizeRetrieve`, `Phenomena/ConfusionErrors`). The multi-digit version is a one-line instance.

One clause, unpacked: the confusion



- **EPAM**: holds for the net learned from $100 + 100$ and $100 + 200$ (checked); every learned net recognizes its own images.
- **ACT-R**: *never* holds, because `pure` is too strong: retrieval is graded, never certain. That is why the theorem doesn't apply to it.

Equal is not low

- The theorem gives *equal* solver use for x and q, not *low* solver use.
- The rate is $(1 - R_q)^n$, with R_q the chance one search of q's associations clears the criterion.
- If $R_q \geq r$, the confused problem reaches the solver with probability at most $(1 - r)^n$.
- With flat associations on q, the same theorem holds, and the errors are not confident.

Here $R_q = 45/46$, $n = 1$: the solver is reached $1/46$ of the time.

Two more consequences of the composition

Whole-problem errors don't depend on the solver (every setting)

- Improving column skill cannot reduce errors from mistaken recognition.

Two sources of error on a confused problem (given the conditions)

- Whole route: always the neighbour's sum.
- Column route: exhaustion $\times$ the solver's own error rate, which comes from the single-digit DOA trials.
- If the other columns are exact, a column error is a single-digit error in place, off by $10^j\delta$. But columns can err together, errors can carry, and they can even cancel: units 10 and tens 4 still give 250.

Swapping the memory: EPAM versus ACT-R

	EPAM net	ACT-R memory
$150 + 100$ answered 200 by retrieval	$45/46$	$225/529 \approx .43$
$150 + 100$: solver invoked	$1/46$	$383/1058 \approx .36$
solver correct, *given* invoked	$\approx .656$	$\approx .656$
correct overall (invoked *and* correct)	$\approx .014$	$\approx .238$
practiced $100 + 100$: overall errors	$\approx .0073$	$\approx .334$
after also studying $100 + 300$	unchanged	$\approx .616$
memory laws (every setting)	determinacy, certainty, stability	none of them

- The confident-error theorem holds for *any* memory with the three laws.
- For ACT-R it doesn't apply. That doesn't mean ACT-R can't err similarly (it does, gradedly); the exact pattern fails, and at this setting that is proved (`setting_actr_not_pattern`).

One memory for both levels

Column facts from the *same* memory as whole problems: each digit pair $d + e$ is coded as a problem, recognized, and retrieved in the same strength table; fingers and counting if that fails.

- **The old model is a special case**: a pair recognized as itself gives exactly the DOA fact (`memFact_self`), so every earlier theorem carries over.
- **Confident errors one level down**: a pair taken for another states the other's sum (`column_confident_errors`), with no new proof.
- **Noninterference needs a condition**: EPAM's practice stays undisturbed *given* that the problem's digit pairs were learned (`shared_learning_preserves_trial`). The old theorem held the solver fixed; here learning could change it.

One memory: the numbers

Memory studies $0+0$, $5+0$, $1+1$, then $100+100$, $100+200$.

	EPAM	ACT-R, separate table	ACT-R, one memory
$0+0$ answered 5 (retrieves $5+0$)	0	0	120/2167
$150+100$: solver invoked	1/46	$\approx .407$	$\approx .407$
solver correct, *given* invoked	$\approx .656$	$\approx .656$	$\approx .385$
correct overall	$\approx .014$	$\approx .267$	$\approx .157$
errors on practiced $100+100$	$\approx .0073$	$\approx .334$	$\approx .386$

- The middle column is a control: same five items in memory, but the solver doesn't use it.
- Extra items alone: *more* correct ($.238 \rightarrow .267$). Sharing the memory: fewer (.157).
- ACT-R without retrieval as use; finger counting is an implausible column fallback.

What each result rests on

Kind	Examples
Library facts	the bridge; the composed trial is a probability distribution
Compositional deductions (every setting)	whole-problem errors independent of the solver; solver use is $(1 - R_q)^n$
Conditional	confident errors; two sources of error; any memory with three laws
At chosen settings	$45/46$, $1/46$, $\approx .656$, $\approx .014$, the ACT-R column

None of these is yet an empirical finding about people. The numbers are illustrations, not fits.

Checking the formalization: a blind simulation

- A prose specification, no Lean; preregistered checks; a fresh agent wrote a simulator from the prose alone.
- **Agreement**: every check whose hypothesis arose passed; 243 frequencies matched Lean exactly.
- **Found what proof didn't**:
  - five theorems' hypotheses were *unrealized* in the tested learning histories (not shown unreachable);
  - a gap at zero total strength;
  - a predicted learning sequence depends on an unstated assumption: a new problem's initial associations.

Model verification and theoretical deduction

- **Simulation** samples executions.
- **Probabilistic model checking** (PRISM; Soar to PRISM, Ganeriwala et al. 2026) analyzes a specified state-transition model exhaustively; it is formal verification, and can treat parameters symbolically.
- **Deductive proof** (CogLean) establishes results for whole classes of models satisfying theoretical assumptions.

The contrast is *model verification* versus *theoretical deduction*. Complementary, not competitors.

CogLean's claim is not formal methods (see the prior work), but the combination: a shared, cumulative library with reusable proofs, cross-theory composition, and deductive discovery.

Relationship to cognitive architectures

- **Complementary**, not an alternative to Soar, ACT-R, or the Common Model of Cognition.
- Architectures accumulate shared mechanisms and models; CogLean tries to accumulate **proved results** about mechanisms, including over classes of them. An architecture could carry such results itself.
- Example: confident errors hold for every memory with three laws (EPAM has them; ACT-R's memory has none). The laws are sufficient, not shown necessary; no general equivalence or divergence of architectures is claimed.
- A possible **formal foundation** for expressing, composing and reasoning about their mechanisms and architectural commitments.
- The present example composes locally, for one task, without committing to a general architecture.
- Longer term: formalize **architectural constraints themselves**, so their implications can be proved and architectures compared in one framework.

What a proof shows, and what it doesn't

- Lean guarantees **validity**: conclusions follow from premises.
- Not **empirical adequacy**: whether the premises describe people.
- Not **explanation**: whether the premises already contain the phenomenon.
- Example: a serial-position result whose mechanism was a bowed schedule. Valid, nearly empty. Now derived from an attention rule, which also gives the isolation effect.
- An explicit audit sorts results as definitional, close, derived, compositional.

What CogLean is, and isn't

It doesn't

- generate hypotheses or theories: people (or AIs) propose them;
- find phenomena in data, or fit parameters;
- show that a theory is true of people;
- decide whether a derivation is an explanation;
- replace simulation or experiment.

It does

- check whether a proposed consequence follows, and from exactly what;
- find counterexamples to informal claims (peaking);
- expose unstated assumptions and conditions;
- say for which class of models a result holds;
- let proved results be reused across theories and tasks.

"Discovery" here means finding out what does, or doesn't, follow from theories people propose.

Limitations, and next

Limitations

- Strengths chosen, not learned; sharpness assumed.
- Error-free carrying; independent columns; finger counting as the column fallback.
- ACT-R without retrieval as use.
- Nonnegativity proved for this model, not library-wide.
- Scalability: exact finite distributions.
- No contact with data yet.

Next

- Derive associations from learning histories, so sharpness is a theorem.
- Learning across whole trials: prove the learning sequence.
- Restate the unrealized predictions in terms of tested features.
- Retrieval that changes memory, carried through the solver.
- Architectural constraints; other domains.

Take-home

- Cognitive theories can be stated once, in a shared formal library, with their consequences proved.
- Independently formalized theories compose, and so do their **theorems**.
- Composition derives consequences none of the parts states; formalization exposes what the theories didn't say.
- Every result is labelled: library fact, compositional deduction, conditional, or illustrative.

The goal: a cumulative theoretical science, in which cognitive constructs and their proved properties are reused across models, tasks and traditions.

`https://github.com/jeffshrager/CogLean`